HACKATHON SCIA 2026 : IA AGENTIQUE AU SERVICE DE L'ÉDUCATION

Le cabinet des identités perdues

Théodore Deguest (theodore.deguest@epita.fr)

Dépôt : github.com/touikss/cabinet-des-identites-perdues

Le cadrage du hackathon annonce un seul critère de jugement : **l'innovation**, « l'audace de l'idée et l'originalité de l'architecture agentique », sans benchmark ni métrique imposée. Ce rapport suit cette logique : l'architecture multi-agents et son originalité d'abord (sections 2 à 4), les détails techniques ensuite.

1. Présentation du projet

« Le cabinet des identités perdues » met le joueur dans la peau d'un visiteur admis à consulter les archives d'un fonds ancien. Dans ce fonds, certains dossiers ont perdu un élément essentiel : le nom attaché à l'une des fiches qu'ils contiennent. Le joueur ouvre un de ces dossiers et se retrouve face à trois dépositions de témoins d'un même événement historique, consignées à l'époque mais jamais authentifiées. Sa tâche, en tant que visiteur de la salle de consultation, est de les interroger pour rendre son nom à la fiche manquante et permettre au dossier de retrouver sa place dans le registre.

Concrètement : le joueur est plongé dans un événement historique réel sans aucun repère, ni nom, ni époque, ni lieu. Trois témoins de cet événement peuvent être interrogés librement, en texte libre. Deux sont des figures anonymes, restées telles dans les archives. Le troisième est en réalité un personnage historique connu, qui a vécu l'événement à la première personne mais dont l'identité s'est perdue dans le dossier, et qui ne dira jamais lui-même qui il est.

Le joueur ne sait pas non plus lequel des trois c'est. Il doit le déterminer uniquement à partir de ce que racontent les témoins, en recoupant des indices répartis entre eux : aucune déposition seule ne suffit à tout comprendre. Une fois prêt, il remplit un « bulletin de résolution » : le nom de l'événement, sa date, l'identité du personnage caché, et lequel des trois témoins c'est. Un score sur 100 points, calculé par le code, sanctionne la précision de la déduction et le nombre de questions posées.

Le rapport avec l'éducation : plutôt que de lire un cours sur un événement historique, le joueur le reconstitue en menant l'enquête, il pose des hypothèses, les teste, les affine. Le niveau visé est plutôt exigeant, pensé pour des lycéens plutôt que pour un jeune public : les témoins restent volontairement avares en indices directs, et la déduction demande de recouper plusieurs fragments plutôt que de deviner sur un seul indice évident. Le thème est libre ou tiré d'un pool d'événements classés par niveau (des dossiers plus ou moins difficiles à percer), ce qui rend le jeu rejouable aussi bien seul qu'en classe.

2. **Architecture agentique**

Quatre rôles d'agents, chacun avec son modèle et sa tâche, organisés autour d'un journal de partie partagé.



2.1. L'Orchestrateur (Claude Sonnet 5)

À partir d'un thème choisi par le joueur (un carton de difficulté, ou une commande sur mesure), il génère en une passe un scénario JSON complet : l'événement réel et sa date exacte, une ambiance d'ouverture neutre, la liste des noms propres interdits, et les trois témoins : étiquette interne, épithète publique (ex. « l'homme au regard perçant »), identité réelle, et system prompt de personnage.

En testant, on a remarqué que le modèle choisissait presque toujours le même témoin (le troisième) comme personnage important, quel que soit le thème, un biais positionnel classique des LLM. Plutôt que de compter sur le prompt pour corriger ça, on a déplacé la décision côté code : le tirage du personnage important se fait en Python, imposé ensuite à l'Orchestrateur, et le champ est réécrit après coup par sécurité. L'équité du jeu ne repose pas sur l'espoir qu'un LLM soit uniformément aléatoire.

2.2. Les agents Témoins (Claude Haiku 4.5)

Chaque témoin a son propre system prompt, généré sur mesure par l'Orchestrateur, et répond toujours à la première personne, en une phrase courte, sans jamais dire de nom propre ni de date exacte. Les règles :

- Aucun témoin ne ment, mais chacun ne connaît que sa propre expérience, ce qui suffit à rendre le dialogue crédible.
- À une question vague ou méta (« que s'est-il passé ? »), pas de résumé : juste un détail personnel minime et peu exploitable.
- À une question précise (un objet, un son, un geste), un vrai fragment d'information, reformulé sans nom propre.
- Face à une hypothèse précise du joueur (« l'ennemi est-il anglais ? », ou pour le personnage important « portez-vous du rouge ? »), le témoin confirme ou infirme honnêtement le fait, avec parfois un indice indirect en plus, sans jamais de mur total, et surtout jamais de mensonge pour se protéger.
- Aucune didascalie, sous aucune forme (ni « *je baisse les yeux* », ni « je détourne le regard... » en prose) : que la parole du personnage.

Les témoins non interrogés peuvent aussi réagir spontanément à ce qui vient d'être dit (confirmer, nuancer, s'étonner), ce qui donne au dialogue une allure de vraie scène à trois plutôt qu'une suite de questions-réponses.

2.3. Le Greffier (Claude Haiku 4.5)

Après chaque réplique, cet agent relit toutes les dépositions et réécrit un carnet de déduction en quatre sections (indices généraux, puis un par témoin). Une règle lui interdit de confondre un geste d'esquive avec un fait observé, un piège rencontré en test : un témoin décrivant vaguement son évitement du regard avait été pris à tort pour un indice sur le décor par une première version du Greffier.

2.4. Le Juge (Claude Sonnet 5)

À la clôture, il compare les réponses du joueur à la vérité du scénario sur trois axes : événement correct, écart en années sur la date, personnage correct. Il ne calcule pas de score global : ce calcul est retiré du LLM (section 3).

3. Le barème : pas de score calculé par un LLM

Le score est calculé en Python à partir des seules évaluations du Juge :

Critère	Points	Règle
Événement historique	40	Tout ou rien.
Époque	20	Par paliers selon l'écart en années : 20 si exact, 15 si ≤10 ans, 10 si ≤50 ans, 5 si ≤100 ans, 3 si ≤200 ans, 0 au-delà.
Personnage important	15	10 pts pour le nom exact + 5 pts pour le bon témoin désigné, comptés indépendamment.
Économie de questions	25	Plein tant que ≤6 questions, puis -2 pts par question supplémentaire.

Un LLM peut juger avec nuance si une réponse floue est correcte. Mais un score de jeu doit être reproductible et vérifiable, on ne l'a donc jamais laissé à la charge du modèle.

4. Itérations sur de vraies parties jouées

Chaque partie est journalisée intégralement (un fichier JSONL par partie, un événement par ligne : appel à chaque agent, réponse brute, réponse affichée après nettoyage). Ça a permis de corriger le système à partir de ce qu'il faisait vraiment, pas seulement de sa conception sur le papier. Trois cas trouvés et traités pendant le hackathon :

Des didascalies passaient malgré la consigne du prompt, sous une forme non prévue par le filtre initial : un préambule à la troisième personne suivi d'un deux-points avant la réplique (« L'archer hésite, puis lâche d'une voix rauque : ... »). Ajout d'un filet de sécurité côté code, en plus du prompt : une fonction retire ce type de préfixe narratif quoi qu'il arrive, sans compter sur le modèle pour suivre la règle à la lettre.

Sur une partie où le personnage caché était Richelieu, le témoin a répondu à « êtes-vous ecclésiastique ? » en employant lui-même le mot « cardinal », un terme non couvert par la liste des noms interdits (qui ne bloquait que

la forme complète « cardinal de Richelieu »), mais très identifiant tout seul. Ça montre une limite des filtres par correspondance exacte de texte ; le correctif (étendre la génération des noms interdits aux titres isolément identifiants) reste à faire, voir section 6.

Le cas le plus intéressant : à la question composée « êtes-vous principal ministre de Louis XIII et portez-vous du rouge ? », ce même témoin a répondu « Je ne porte pas de rouge », un mensonge pur, alors que la règle de base du jeu est qu'aucun témoin ne ment. On a renforcé le prompt de l'Orchestrateur avec une règle explicite : face à une question précise et vérifiable sur un trait du personnage important (vêtement, titre, fonction), le témoin doit confirmer ou infirmer honnêtement, quitte à minimiser juste après, mais sans jamais nier un fait vrai. Rejoué sur un scénario équivalent après correctif, la même question donne :

> *« Je ne suis point le principal ministre, mais seulement celui qui s'occupe de certaines affaires de lettres et de langue, et quant à la couleur que je porte, oui, c'est exact, mais ce détail importe peu auprès de l'essentiel. »*

Sans ce genre de garantie, une piste juste du joueur ne peut jamais être confirmée, et toute la mécanique de déduction perd son intérêt.

Un extrait d'une partie jouée jusqu'au bout (score final 100/100, thème « la fondation de l'Académie française », personnage caché : Richelieu, désigné publiquement comme « l'homme au regard perçant ») :

> QUESTION : de quelle type de charge s'agit-il et combien êtes-vous à la porter
>
> L'HOMME AU REGARD PERÇANT
>
> *« Nous sommes chargés de veiller à ce que la langue demeure pure et noble, ni corrompue par l'usage ni appauvrie par l'oubli, nous étions une vingtaine au commencement, guère plus. »*
>
> *[réaction spontanée d'un autre témoin]*
>
> *« Je me souviens que nous étions bien moins nombreux quand nous nous réunissions sans témoin, et que chacun parlait plus librement. »*

Chaque réponse donne un vrai fragment (une fonction, un nombre) sans jamais rien nommer, et la réaction du deuxième témoin arrive sans que le joueur ait eu à la demander. En croisant ce genre d'indices, le joueur de cette partie a retrouvé l'événement, la date exacte et le personnage caché en six questions, score maximal sur les quatre axes du barème.

5. **Architecture technique**

Backend FastAPI qui expose la logique ci-dessus en JSON et sert aussi le frontend statique. Les sessions vivent en mémoire, sans base de données, raisonnable pour un format de 48h.

Les agents Témoins, Greffier et Juge appellent le modèle les uns après les autres, réplique par réplique. Plutôt que d'attendre la fin de toute une séquence avant de répondre au client, le serveur diffuse chaque réplique dès qu'elle est produite, via un flux NDJSON que le frontend affiche au fur et à mesure. Le joueur voit les témoins parler l'un après l'autre en temps réel, au lieu d'un bloc de texte qui tombe d'un coup après un silence.

Le frontend est en HTML/CSS/JavaScript sans framework ni étape de build, avec une identité visuelle « cabinet d'archives » (EB Garamond et IBM Plex Mono, palette parchemin, tampons et fiches cartonnées) pour appuyer le rôle d'enquêteur.

6. Limites et suite possible

- Les parties en cours sont perdues au redémarrage du serveur (stockage en mémoire uniquement). Une persistance légère serait la première suite logique.
- Pour un usage réel en classe, un suivi de progression par élève (thèmes déjà vus, score moyen, types d'événements à retravailler) apporterait plus de valeur que le jeu isolé.
- Un mode enseignant pourrait générer des dossiers alignés sur un programme scolaire précis plutôt que sur un thème libre.
- La génération des noms interdits devrait aussi couvrir les titres et fonctions isolément identifiants, pas seulement les noms propres complets (cas du mot « cardinal », section 4).

7. Comment tester l'application

Prérequis : Python 3.13+, l'outil « uv » (docs.astral.sh/uv), et une clé API Anthropic valide.

```
git clone https://github.com/touikss/cabinet-des-identites-perdues.git
cd cabinet-des-identites-perdues
export ANTHROPIC_API_KEY="votre-clé-api"
uv run uvicorn hackaton.server:app --reload
```

Puis ouvrir *http://localhost:8000/*. « uv run » installe les dépendances tout seul au premier lancement, rien d'autre à faire. La clé peut aussi venir d'*ANTHROPIC_AUTH_TOKEN* ou d'un profil déjà configuré sur la machine, le SDK la trouve automatiquement. Pas de base de données à préparer : les parties vivent en mémoire pendant que le serveur tourne (voir section 6).

8. Conclusion

Quatre agents modestes en apparence (un orchestrateur, trois témoins, un greffier, un juge) suffisent à porter une expérience où l'histoire s'apprend en la questionnant plutôt qu'en la recevant. L'idée de base (un témoin anonyme parmi d'autres qui est en réalité une figure historique, à démasquer par recoupement) est simple à expliquer et pas évidente à deviner d'avance. Le reste du travail, fiabiliser les témoins à partir de vraies parties jouées plutôt que de s'arrêter au premier prompt qui marche, c'est ce qui fait la différence entre un prototype qui marche une fois et un système qu'on peut vraiment donner à jouer à quelqu'un.